

Company : Reliance Retail

Experience : 5 Years

① Q. Tell me about yourself and your experience as a DevOps Engineer.

Interview Answer :

Currently, I am working as an AWS DevOps Engineer at Reliance Retail, with around 5 years of experience in designing, automating and managing cloud infrastructure and CI/CD pipelines.

In my current role, I work on AWS cloud services and Kubernetes (EKS) to deploy and scale microservices based applications. My focus is on automation, reliability, security and performance optimization.

My day usually starts by checking the health of production systems and monitoring dashboards like CloudWatch, Grafana and Prometheus. I review overnight alerts, investigate any incidents and coordinate with the development or operations teams to resolve them.

I work on CI/CD pipelines using Jenkins and GitHub Actions. Whenever developers push code to Git, the pipeline gets triggered, builds the application, runs tests, performs security scans using SonarQube and Trivy, builds Docker images and pushes them to ECR. Finally, the application is deployed to EKS using ArgoCD with blue-green or rolling deployment strategy.

I also manage infrastructure as code using Terraform. I provision and maintain AWS resources like VPC, EKS, ALB, RDS, S3, IAM, Security Groups and Route53 through reusable Terraform modules.

Monitoring and observability is a key part of my role. I set up dashboards, alerts and logs using CloudWatch, Grafana and Prometheus to ensure high availability and quick incident response.

I believe in automation and continuous improvement. I write scripts in Bash and Python to automate repetitive tasks, optimize costs and improve developer productivity. I also follow best practices for security, backup, disaster recovery and compliance.

Overall, I enjoy working in a collaborative DevOps environment where I can bridge the gap between development and operations and deliver reliable and scalable solutions.

Key Responsibilities

- Manage AWS infrastructure (EC2, EKS, RDS, S3, VPC, IAM, ALB, Route53)
- Build and maintain CI/CD pipelines using Jenkins, GitHub Actions
- Deploy microservices on Kubernetes (EKS)
- Implement Infrastructure as Code using Terraform
- Monitor systems using CloudWatch, Grafana, Prometheus and set up alerts
- Troubleshoot production issues and perform RCA
- Ensure security, compliance and cost

Interviewer Follow-up Questions

1. Which AWS services do you use most often?

Ans: I use EKS, EC2, S3, RDS, IAM, ALB, CloudWatch, ECR and Route53 most frequently in my projects.

2. How do you ensure high availability of applications?

Ans: I deploy across multiple AZs, use ALB, Auto Scaling, health checks, RDS Multi-AZ, and monitor with CloudWatch to ensure high availability.

3. How do you handle production incidents?

Ans: I follow a structured approach - identify, triage, mitigate, communicate, perform RCA and implement

② Q. Explain your day-to-day responsibilities as a DevOps Engineer.

Interview Answer :

Currently, I am working as an AWS DevOps Engineer at Reliance Retail. My day usually starts with checking the health of our production environment. I review CloudWatch dashboards, Grafana dashboards and alerts from Prometheus to make sure all the services are running smoothly. If there are any alerts or unusual spikes in CPU, memory, latency or error rates, I investigate and resolve them or coordinate with the concerned team.

I work on CI/CD pipelines using Jenkins and GitHub Actions. Whenever developers push code to Git, the pipeline is triggered. The code is built, unit tested, scanned for vulnerabilities using SonarQube and Trivy, then Docker image is created and pushed to Amazon ECR. Finally, the application is deployed to Kubernetes (EKS) using ArgoCD.

I manage AWS infrastructure as code using Terraform. I create reusable modules for VPC, EKS, RDS, ALB, IAM, Security Groups, S3 and other resources. This helps in maintaining consistent environments across dev, staging and production.

I also monitor application and infrastructure using CloudWatch, Grafana and Prometheus. I create dashboards, set up alarms and SNS notifications to ensure quick response to incidents. I perform RCA for incidents and work with the team to implement fixes and improve system reliability.

I handle deployments, rollbacks, blue-green and canary deployments to ensure zero downtime. I also automate tasks using Bash and Python scripts for backups, cleanups and environment provisioning.

I follow best practices for security, backup, disaster recovery and cost optimization. I regularly review IAM policies, security groups, AWS Config rules and Trusted Advisor recommendations to keep the environment secure and cost efficient.

Key Responsibilities

- Monitor AWS infrastructure and application health using CloudWatch, Grafana, Prometheus
- Build and maintain CI/CD pipelines using Jenkins and GitHub Actions
- Deploy applications on Kubernetes (EKS) using ArgoCD
- Manage infrastructure as code using Terraform
- Automate tasks using Bash and Python scripts
- Handle production incidents and perform RCA
- Implement security best practices and manage IAM, Security Groups, AWS Config
- Optimize costs and ensure high availability and scalability

Interviewer Follow-up Questions

1. Which tools do you use for monitoring?

Ans: I use CloudWatch for infrastructure monitoring, Grafana and Prometheus for application metrics and alerts.

2. How do you ensure zero downtime during deployments?

Ans: I use blue-green or canary deployments and rolling updates in Kubernetes. I also perform health checks before routing traffic.

3. How do you handle high CPU or memory alerts?

Ans: I check the metrics in CloudWatch and Grafana, identify the root cause, optimize the application or scale the resources using Auto Scaling.

Interview Tip

While answering, be specific about the tools and services you use daily.

- ③ Q. You have cloned a Git repository, but while running `git pull`, you get the error: "not a git repository". How would you troubleshoot and resolve this issue?

Interview Answer :

This error occurs when Git cannot find a `.git` directory in the current location or in any of its parent directories. I troubleshoot this issue in a structured way.

First, I verify whether I am in the correct directory. Sometimes we move out of the repository folder and try to run `git pull` from a different location. I run `'pwd'` (Linux/Mac) or `'cd'` (Windows) to check the current path.

Next, I check if the `.git` folder exists in the current directory using `'ls -a'`. If it is missing, the directory is not a Git repository. I navigate to the correct project folder where I originally cloned the repository.

If I am in the correct folder but `.git` is missing, the repository might be corrupted or accidentally deleted. In that case, I re-clone the repository:

```
rm -rf .git
git clone <repository-url>
```

If the `.git` folder exists but the error still comes, I check for permission issues or file system problems. I also validate Git installation using `'git --version'`.

If the remote origin is misconfigured, I verify it using `'git remote -v'`. If it's missing or incorrect, I set it again:

```
git remote add origin <repository-url>
```

Once everything is correct, I run `'git pull'` again. In most cases, these steps resolve the issue.

In production, I follow best practices like not manually deleting `.git`, using branches properly, and always pulling code through CI/CD pipelines to avoid such issues.

Key Responsibilities

- Manage source code using Git and GitHub
- Troubleshoot code pull, push and branching issues
- Ensure secure access and authentication using SSH keys and tokens
- Maintain clean and consistent repositories
- Integrate Git with CI/CD pipelines
- Follow Git best practices for collaboration and version control

Interviewer Follow-up Questions

1. What is the difference between `'git pull'` and `'git fetch'`?

Ans: `git fetch` only downloads changes from remote repository, but does not merge. `git pull` = `fetch` + `merge`.

2. How do you resolve merge conflicts during `git pull`?

Ans: I open the conflicted files, resolve the conflicts manually, stage the changes using `'git add <file>'` and then commit using `'git commit'`.

3. What is the use of `'git remote -v'`?

Ans: It shows the configured remote repositories. It helps verify the remote URL for fetch and push operations.



Interview Tip

When answering, show your problem-solving approach step-by-step. Mention commands

④ Q. How do you provide access to an Amazon S3 bucket? What permissions and policies need to be configured?

Interview Answer :

To provide access to an Amazon S3 bucket, I follow the principle of least privilege. First, I decide who needs access (users, IAM roles or external accounts) and what level of access they require.

For IAM users or roles, I attach the appropriate IAM policy with S3 permissions like `s3:ListBucket`, `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`, etc.

Next, I configure a bucket policy on the S3 bucket to allow or deny access to specific principals. Bucket policy works at the resource level and is used to control cross-account access, public access or specific IP access.

If I need to allow public access, I configure a bucket policy to allow `GetObject` for everyone and disable Block Public Access (only if required for the use case).

For fine-grained access, I use IAM policies and bucket policies together.

I also enable versioning, server-side encryption (SSE-S3 or SSE-KMS) and access logging for security and compliance.

In cross-account scenarios, I use bucket policies to allow specific IAM roles from another AWS account to access the bucket securely.

EXAMPLE - BUCKET POLICY (ALLOW READ ACCESS)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:role/AppReadRole" },
      "Action": [ "s3:GetObject" ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

EXAMPLE - IAM POLICY (ATTACHED TO ROLE/USER)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [ "s3:ListBucket" ],
      "Resource": "arn:aws:s3:::my-bucket"
    },
    {
      "Effect": "Allow",
      "Action": [ "s3:GetObject", "s3:PutObject" ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

KEY PERMISSIONS

- `s3:ListBucket` - List objects in a bucket
- `s3:GetObject` - Read objects from a bucket
- `s3:PutObject` - Upload objects to a bucket
- `s3:DeleteObject` - Delete objects from a bucket
- `s3:ListBucketMultipartUploads` - List multipart uploads

BEST PRACTICES

- ✓ Follow least privilege access.
- ✓ Use IAM roles instead of access keys.
- ✓ Enable versioning and encryption.
- ✓ Enable Block Public Access.
- ✓ Use bucket policies for cross-account access.
- ✓ Enable access logs for audit and monitoring.

COMMON MISTAKES

- ✗ Giving full S3 access (`s3:*`) to users.
- ✗ Making bucket public without requirement.
- ✗ Not enabling versioning or encryption.
- ✗ Using access keys instead of IAM roles.
- ✗ Ignoring Block Public Access settings.

🕒 1-MINUTE ANSWER

I provide access to an S3 bucket by attaching IAM policies to users/roles and configuring bucket policies. IAM policies define what actions are allowed and bucket policies define who can access the bucket. I follow least privilege, enable encryption, versioning and logging for security.

💡 KEY TAKEAWAYS

- Access to S3 is controlled using IAM policies and Bucket policies.
- IAM policies control permissions; Bucket policies control access.
- Always follow least privilege and security best practices.
- Use roles and policies for secure cross-account access.

❓ INTERVIEWER FOLLOW-UP QUESTIONS

1. How do you allow cross-account access to an S3 bucket?

Ans: I use a bucket policy to allow the specific IAM role or user from another AWS account. I specify the principal ARN and allowed actions in the bucket policy.

2. What is the difference between IAM policy and Bucket policy?

Ans: IAM policy is attached to a user/role and defines permissions. Bucket policy is attached to the bucket and defines who can access the bucket and what actions.

3. How do you make an object public in S3?

Ans: I attach a bucket policy to allow `s3:GetObject` for all principals (*). I also ensure Block Public Access is disabled (if public access is required).

- ⑤ Q. An application needs to communicate with an EC2 instance deployed in a private subnet behind a Multi-AZ Load Balancer. How would you design this communication?

Interview Answer :

To enable secure communication with an EC2 instance in a private subnet behind a Multi-AZ Load Balancer, I follow a secure and scalable architecture.

The EC2 instance will be in private subnets across multiple Availability Zones for high availability. A Load Balancer (Application or Network Load Balancer) will be placed in public subnets across multiple AZs and will route traffic to the EC2 instances in the private subnets.

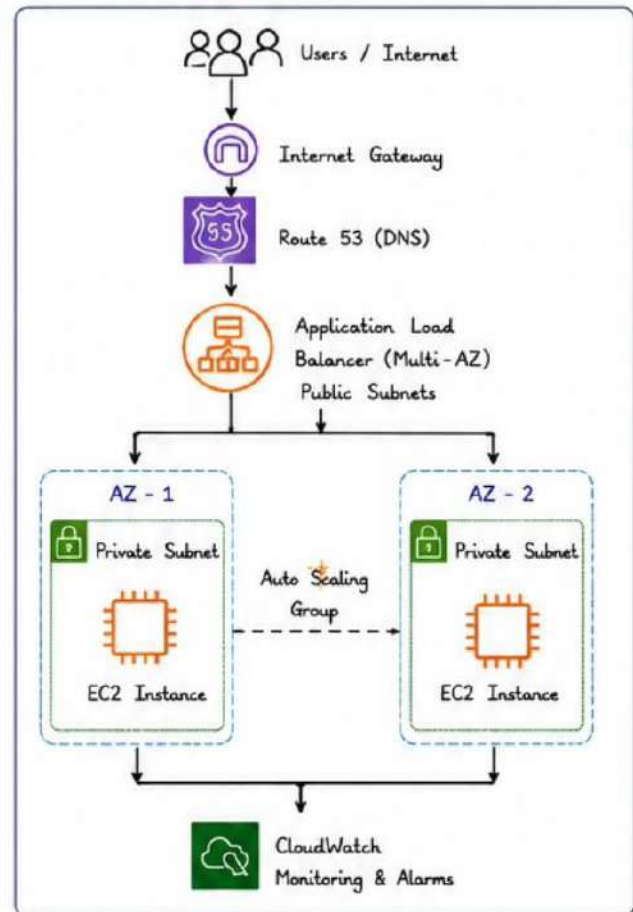
The communication happens through the Load Balancer, which has a security group allowing inbound traffic (e.g., HTTP/HTTPS on 80/443) from trusted sources. The EC2 instances have a security group that allows inbound traffic only from the Load Balancer's security group.

For applications outside AWS (internet users), traffic comes to the Load Balancer's public DNS. The Load Balancer distributes the requests to healthy EC2 instances in private subnets.

For internal communication (other AWS services or VPC resources), I use a VPC Endpoint (if needed) or connect through a private Application Load Balancer / Internal NLB for secure private access.

All data in transit is encrypted using SSL/TLS. I also enable CloudWatch for monitoring, set up alarms and use Auto Scaling for handling traffic spikes.

ARCHITECTURE DIAGRAM



DESIGN COMPONENTS

- EC2 Instances in Private Subnets (Multi-AZ)
- Application Load Balancer in Public Subnets
- Security Groups for ALB and EC2
- Route 53 for DNS Resolution
- Internet Gateway for public access to ALB
- Auto Scaling Group for EC2
- SSL/TLS for encrypted communication
- CloudWatch for monitoring and alarms

BEST PRACTICES

- ✓ Use Multi-AZ for high availability
- ✓ Keep EC2 in private subnets for security
- ✓ Allow traffic to EC2 only from ALB SG
- ✓ Use SSL/TLS for all client communications
- ✓ Enable Health Checks on ALB
- ✓ Use Auto Scaling to handle traffic spikes
- ✓ Monitor with CloudWatch and set alarms

COMMON MISTAKES

- ✗ Exposing EC2 instances directly to Internet
- ✗ Allowing 0.0.0.0/0 to EC2 security group
- ✗ Not using Multi-AZ for Load Balancer
- ✗ Ignoring health checks on ALB
- ✗ Not enabling monitoring and alarms
- ✗ Not using Auto Scaling for variable load

🕒 1-MINUTE ANSWER

I place EC2 instances in private subnets across multiple AZs for high availability and security. An Application Load Balancer in public subnets distributes incoming traffic to the EC2 instances. Security groups allow inbound traffic to ALB from trusted sources and allow traffic from ALB to EC2. I use Route 53 for DNS, SSL/TLS for encryption, Auto Scaling for handling traffic spikes and CloudWatch for monitoring and alerts.

💡 KEY TAKEAWAYS

- Use public ALB + private EC2 for secure and scalable design.
- ALB handles traffic distribution and high availability.
- Security groups control access between ALB and EC2.
- Multi-AZ and Auto Scaling ensure reliability and scalability.
- Monitoring and alarms help in quick issue detection.

❓ INTERVIEWER FOLLOW-UP QUESTIONS

1. Why do we put EC2 in private subnets?

Ans: To prevent direct access from the internet and protect the instances from attacks. Traffic comes only through the Load Balancer.

2. Why use Application Load Balancer instead of Network Load Balancer?

Ans: ALB works at Layer 7 and supports HTTP/HTTPS, path-based routing, host-based routing and advanced features. NLB works at Layer 4.

3. How do you handle traffic spikes?

Ans: I use Auto Scaling Group for EC2 instances behind the ALB. Based on CPU utilization or request count, new instances are launched automatically.

6 Q. How do you implement CI/CD for a microservices application using Docker, Jenkins and Kubernetes? Explain the flow in detail.

Interview Answer :

To implement CI/CD for a microservices application, I use Jenkins for automation, Docker for containerization and Kubernetes for deployment and orchestration.

Developers push code to GitHub/Bitbucket. A webhook triggers the Jenkins pipeline. Jenkins pulls the code, runs unit tests, builds the Docker image and scans it for vulnerabilities.

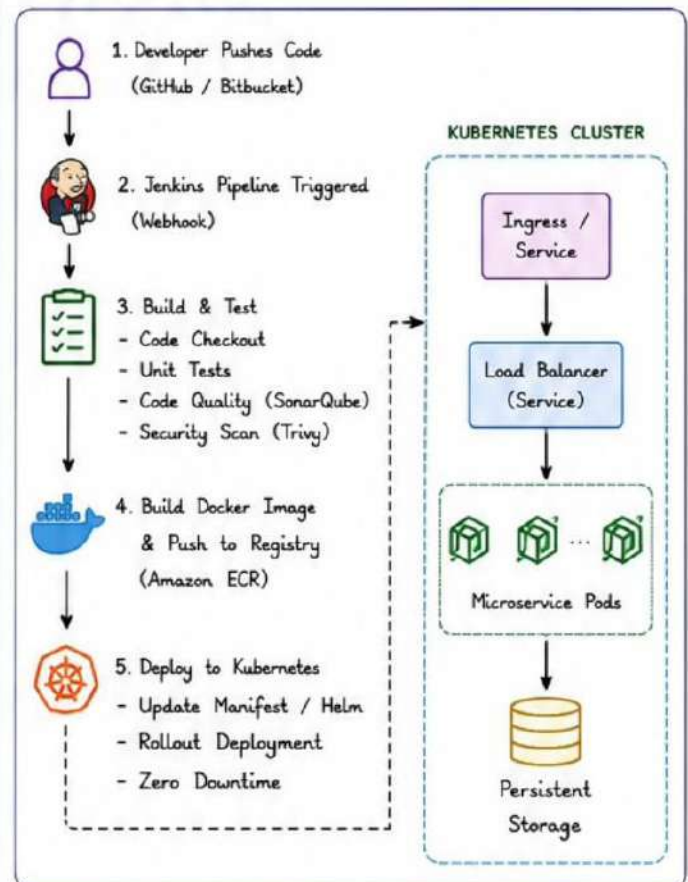
If the build is successful, the image is pushed to a container registry like Amazon ECR.

For CD, Jenkins updates the Kubernetes manifests (Helm charts or Kustomize) with the new image tag and deploys to the target environment (dev/ staging / prod). Kubernetes pulls the new image and rolls out the update with zero downtime.

I use GitOps approach with ArgoCD in production for continuous delivery. It watches the Git repo for changes and syncs the desired state to Kubernetes.

I also implement notifications, rollback strategy and environment promotion with approval gates for production releases.

CI/CD FLOW DIAGRAM



KEY COMPONENTS

- | | |
|----------------------|-----------------------|
| • GitHub / Bitbucket | - Source Code |
| • Jenkins | - CI/CD Automation |
| • Docker | - Containerization |
| • Amazon ECR | - Container Registry |
| • Kubernetes (EKS) | - Orchestration |
| • Helm / Kustomize | - Manifest Management |
| • ArgoCD (GitOps) | - Continuous Delivery |
| • SonarQube, Trivy | - Quality & Security |

BEST PRACTICES

- ✓ Keep pipeline fast and feedback early
- ✓ Run unit & integration tests in pipeline
- ✓ Scan images for vulnerabilities
- ✓ Use immutable Docker images
- ✓ Use Helm charts for deployments
- ✓ Implement Blue-Green / Canary deployments
- ✓ Use GitOps for production environments
- ✓ Monitor with Prometheus & Grafana

COMMON MISTAKES

- ✗ Hardcoding image tags
- ✗ Skipping unit / integration tests
- ✗ Not scanning images for vulnerabilities
- ✗ Using latest tag in production
- ✗ No rollback or deployment strategy
- ✗ Manual changes in Kubernetes (drift)
- ✗ Not monitoring application health

1-MINUTE ANSWER

I implement CI/CD for microservices using Jenkins, Docker and Kubernetes. Code push triggers Jenkins pipeline which builds, tests and scans the code. Docker image is built and pushed to ECR. Jenkins then updates Kubernetes manifests and deploys the application. Kubernetes pulls the new image and rolls out the update with zero downtime. I follow GitOps with ArgoCD for continuous delivery.

KEY TAKEAWAYS

- CI ensures code quality through automated build, test and scan.
- CD ensures fast and reliable delivery to Kubernetes.
- Containers provide consistency across environments.
- Kubernetes provides scalability, self-healing and zero downtime.
- GitOps with ArgoCD ensures auditability and consistency.

INTERVIEWER FOLLOW-UP QUESTIONS

1. How do you handle rollback in production?

Ans: I use Kubernetes rolling update with revision history. If the new version fails health checks, I rollout undo to the previous stable version. In Helm, I use 'helm rollback <release> <rev>'.

2. How do you manage different environments?

Ans: I use separate namespaces or clusters for dev, staging and prod. I manage configuration using Helm values files or Kustomize overlays and store secrets in AWS Secrets Manager.

3. How do you ensure zero downtime?

Ans: I use rolling updates, readiness and liveness probes in Kubernetes. Traffic is routed only to healthy pods, ensuring zero downtime during deployments.

7 Q. How do you handle secrets and sensitive data in a DevOps environment? Which tools and best practices do you follow?

Interview Answer :

Handling secrets securely is critical in DevOps. I never store secrets in code, configs or Docker images. I always use dedicated secret management tools with proper access control, encryption and audit.

In my current projects, I use AWS Secrets Manager and HashiCorp Vault to store and manage secrets like DB passwords, API keys, third-party tokens, SSH keys etc.

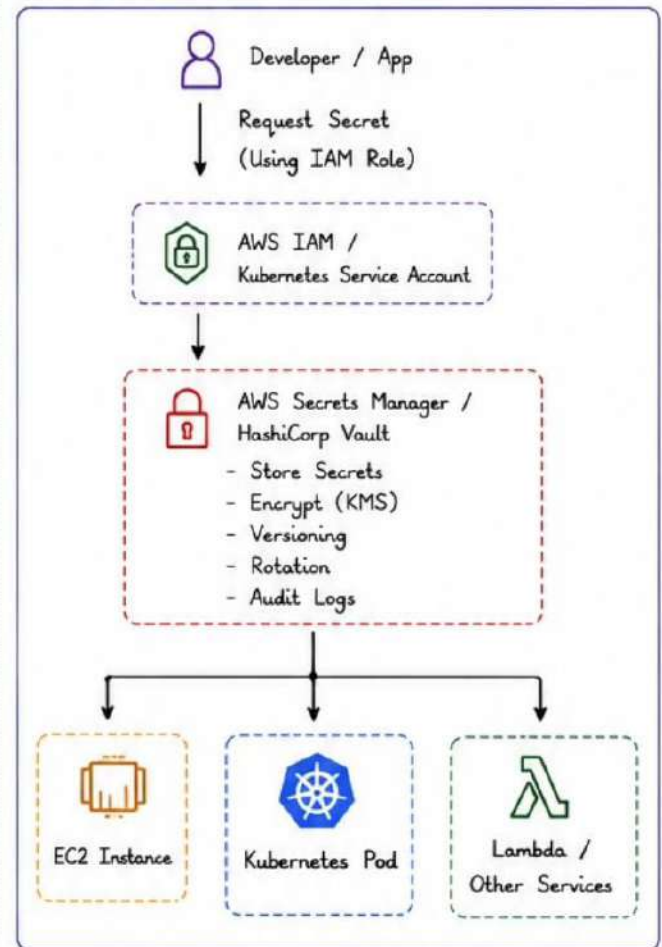
Applications access secrets at runtime using IAM roles (IRSA in EKS) instead of hardcoding them. Secrets are encrypted at rest (KMS) and in transit (TLS).

I follow least privilege access. Only required applications or services can access specific secrets. I enable versioning, rotation and audit logs for all secrets.

In CI/CD pipelines, I avoid printing secrets in logs and use masked variables. For Kubernetes, I use External Secrets Operator to sync secrets from AWS Secrets Manager / Vault into Kubernetes Secrets.

This approach reduces the risk of credential leakage and ensures compliance and security best practices.

SECRETS MANAGEMENT ARCHITECTURE



TOOLS I USE

- AWS Secrets Manager
- HashiCorp Vault
- AWS KMS (Encryption)
- IAM Roles / IRSA (EKS)
- External Secrets Operator (K8s)
- SOPS (For GitOps encryption)
- Parameter Store (SSM)
- GitHub Actions Secrets (CI/CD)

BEST PRACTICES

- ✓ Never store secrets in code or images
- ✓ Use dedicated secret management tools
- ✓ Follow least privilege access
- ✓ Enable encryption at rest and in transit
- ✓ Rotate secrets regularly
- ✓ Use IAM roles instead of access keys
- ✓ Audit and monitor secret access
- ✓ Mask secrets in CI/CD logs

COMMON MISTAKES

- ✗ Hardcoding secrets in code or configs
- ✗ Committing .env or config files
- ✗ Using static access keys
- ✗ Over-privileged IAM policies
- ✗ Not rotating secrets
- ✗ Printing secrets in logs
- ✗ Storing secrets in plain text

1-MINUTE ANSWER

I handle secrets using AWS Secrets Manager / HashiCorp Vault. I never store secrets in code. Applications access secrets at runtime using IAM roles. I follow least privilege, enable encryption, rotation and audit logs. In Kubernetes, I use External Secrets Operator to sync secrets securely.

KEY TAKEAWAYS

- Security of secrets is as important as the application itself.
- Use dedicated tools like Secrets Manager or Vault.
- IAM roles + least privilege = secure access.
- Encrypt secrets and rotate regularly.
- Monitor and audit secret access for compliance.

INTERVIEWER FOLLOW-UP QUESTIONS

1. How do you rotate secrets in AWS Secrets Manager?

Ans: I enable automatic rotation using Lambda function. Secrets Manager creates a new version, updates the secret in the target service (like RDS) and then marks the new version as AWSCURRENT.

2. How do you manage secrets in Kubernetes?

Ans: I use External Secrets Operator to sync secrets from AWS Secrets Manager / Vault into Kubernetes Secrets. Pods access secrets using service accounts (IRSA) instead of static keys.

3. What is the difference between Secrets Manager and Parameter Store?

Ans: Secrets Manager is designed for secrets with rotation, versioning and audit. Parameter Store is for configuration data and does not support automatic rotation.

8 Q. How do you monitor and troubleshoot a production application in a DevOps environment ?

Interview Answer :

Monitoring and troubleshooting in production is all about early detection, quick analysis and fast resolution.

I follow a proactive monitoring approach using CloudWatch, Prometheus and Grafana to monitor application health, infra metrics, logs and user experience.

I set up dashboards and alarms on key metrics like CPU, Memory, Disk, Response Time, Error Rate, Request Count, 5xx errors, etc. Alarms are integrated with SNS, Slack and PagerDuty for instant notifications.

For logs, I centralize all application and server logs using ELK Stack (Elasticsearch, Logstash, Kibana) or CloudWatch Logs. This helps in searching, filtering and analyzing logs quickly during issues.

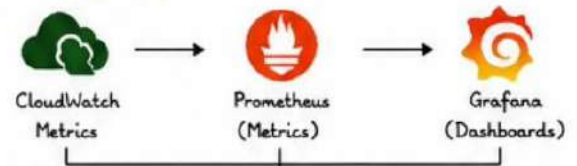
When an issue occurs, I follow a systematic approach: Identify the alert → Check dashboards → Analyze logs → Find root cause → Fix and deploy → Monitor again.

I also use AWS X-Ray for tracing requests and identify bottlenecks in microservices.

Post-incident, I do RCA (Root Cause Analysis) and update runbooks to prevent repeat issues.

MONITORING & TROUBLESHOOTING FLOW

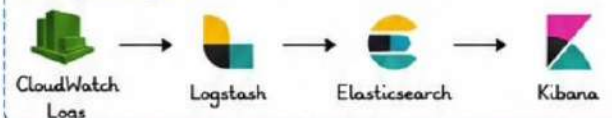
1. Monitoring Setup



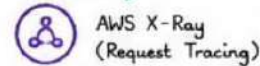
2. Alerting



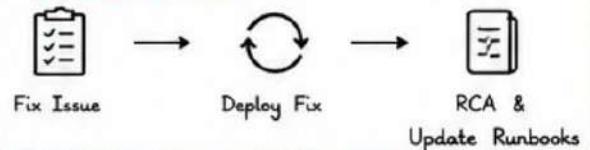
3. Log Management



4. Tracing



5. Resolution & RCA



TOOLS I USE

- AWS CloudWatch - Metrics, Logs, Alarms
- Prometheus - Metrics Collection
- Grafana - Dashboards & Visualization
- ELK Stack - Logs (Elasticsearch, Logstash, Kibana)
- AWS X-Ray - Request Tracing
- Slack / SNS - Notifications
- PagerDuty - On-call Alerting
- SSM / Session Manager - Troubleshooting Access

BEST PRACTICES

- ✓ Monitor key metrics and set appropriate alarms
- ✓ Use dashboards for real-time visibility
- ✓ Centralize logs for faster troubleshooting
- ✓ Use tracing for microservices performance
- ✓ Keep runbooks and SOPs ready
- ✓ Perform RCA for every major incident
- ✓ Automate recovery where possible
- ✓ Test alerts and monitoring regularly

COMMON MISTAKES

- ✗ No monitoring or alerting in place
- ✗ Too many alerts (alert fatigue)
- ✗ Not centralizing logs
- ✗ Ignoring small issues
- ✗ No dashboards for visibility
- ✗ No runbooks or documentation
- ✗ Not doing RCA after incidents
- ✗ No tracing in microservices

1-MINUTE ANSWER

I monitor production using CloudWatch, Prometheus and Grafana for metrics, ELK or CloudWatch Logs for logs and AWS X-Ray for tracing. Alarms notify via SNS, Slack and PagerDuty. When an issue occurs, I check dashboards, analyze logs, find root cause, fix and deploy the solution. Post-incident, I perform RCA and update runbooks to prevent future issues.



KEY TAKEAWAYS

- Proactive monitoring helps detect issues early.
- Centralized logs + tracing = faster root cause analysis.
- Alerts must be actionable and meaningful.
- Dashboards provide real-time visibility.
- RCA and runbooks improve system reliability.

INTERVIEWER FOLLOW-UP QUESTIONS

1. How do you reduce alert fatigue ?

Ans: I create meaningful alerts on critical metrics only. I set proper thresholds, use alarm aggregation and suppress duplicate alerts. I also review and remove unused alerts regularly.

2. How do you troubleshoot high CPU usage ?

Ans: I check CloudWatch metrics, identify the instance, connect using SSM, check top processes (top/http), analyze logs and fix the root cause.

3. Why is centralized logging important ?

Ans: Centralized logging helps in searching across all services, faster root cause analysis, audit and compliance and also helps in trend analysis and monitoring.

9 Q. What is Infrastructure as Code (IaC) ?

How do you implement it and what are its benefits ?

Interview Answer :

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure through machine-readable code instead of manual processes. IaC makes infrastructure consistent, repeatable, version-controlled and automated.

I use tools like Terraform, AWS CloudFormation and Ansible to implement IaC in my projects.

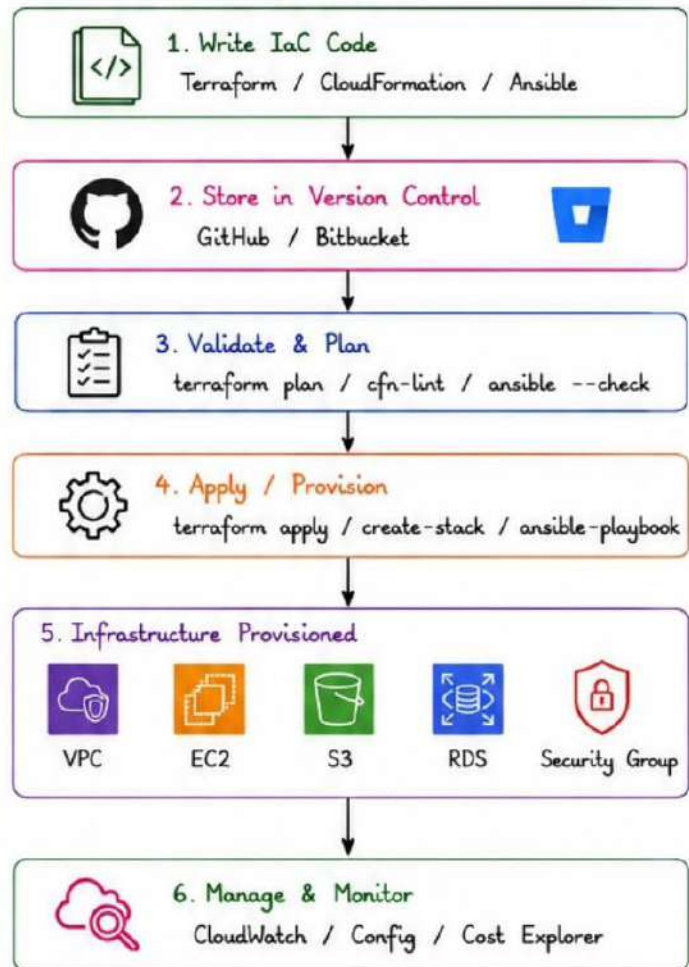
How I Implement IaC :

- **Define** - Write infrastructure code using Terraform / CloudFormation (e.g., VPC, EC2, RDS, S3).
- **Version Control** - Store code in Git (GitHub / Bitbucket).
- **Validate** - Run plan / validate to check for errors.
- **Provision** - Apply the code to create/update infrastructure.
- **Manage State** - Use remote state (S3 + DynamoDB in Terraform) for state locking and consistency.
- **Automate** - Integrate IaC in CI/CD pipeline for automatic provisioning and changes.

Benefits :

- ✓ **Consistency** - Same code creates same environment every time.
- ✓ **Version Control** - Track changes, rollback if needed.
- ✓ **Automation** - Faster provisioning and easy scaling.
- ✓ **Collaboration** - Team can review, share and reuse code.
- ✓ **Cost Efficient** - Reduce manual errors and wasted resources.
- ✓ **Audit & Compliance** - Code and changes are auditable.

IaC IMPLEMENTATION FLOW



TOOLS I USE

- Terraform - Multi-Cloud IaC Tool
- AWS CloudFormation - AWS Native IaC
- Ansible - Configuration Management
- AWS S3 + DynamoDB - Remote State Backend
- Git / GitHub - Version Control
- Terragrunt - Manage Terraform at Scale
- Checkov / TFLint - IaC Static Code Analysis

BEST PRACTICES

- ✓ Keep IaC code modular and reusable
- ✓ Store state files in remote backend
- ✓ Use workspaces or environments (dev/stage/prod)
- ✓ Run plan in pipeline before apply
- ✓ Protect state files with encryption & locking
- ✓ Use tagging for all resources
- ✓ Review code through Pull Requests
- ✓ Automate destroy for non-prod environments

COMMON MISTAKES

- ✗ Hardcoding values in code
- ✗ Committing state files in repo
- ✗ Not using remote state
- ✗ Ignoring code review
- ✗ Over-provisioning resources
- ✗ Not using variables & modules
- ✗ Manual changes to IaC resources
- ✗ No rollback strategy

1-MINUTE ANSWER

IaC means managing infrastructure using code. I use tools like Terraform, CloudFormation and Ansible. I write code, store in Git, validate, apply it to provision infrastructure and manage state remotely. IaC provides consistency, automation, version control, collaboration, cost efficiency and helps in audit & compliance.

KEY TAKEAWAYS

- Infrastructure is managed using code, not manual steps.
- IaC makes infrastructure consistent, repeatable and scalable.
- Tools like Terraform, CloudFormation and Ansible are widely used.
- Always store state remotely and follow security best practices.
- IaC should be part of CI/CD for full automation.

INTERVIEWER FOLLOW-UP QUESTIONS

1. What is the difference between Terraform and CloudFormation?

Ans: Terraform supports multi-cloud (AWS, Azure, GCP, etc.) with one language (HCL). CloudFormation is AWS-specific and uses JSON/YAML templates.

2. What is Terraform state file and why is it important?

Ans: Terraform state file stores the current state of infrastructure. It helps Terraform understand what is already created and what changes are needed.

3. How do you handle secrets in IaC?

Ans: I use AWS Secrets Manager or SSM Parameter Store to store secrets and reference them in code instead of hardcoding.

10 Q. Explain the process of upgrading a Kubernetes cluster.

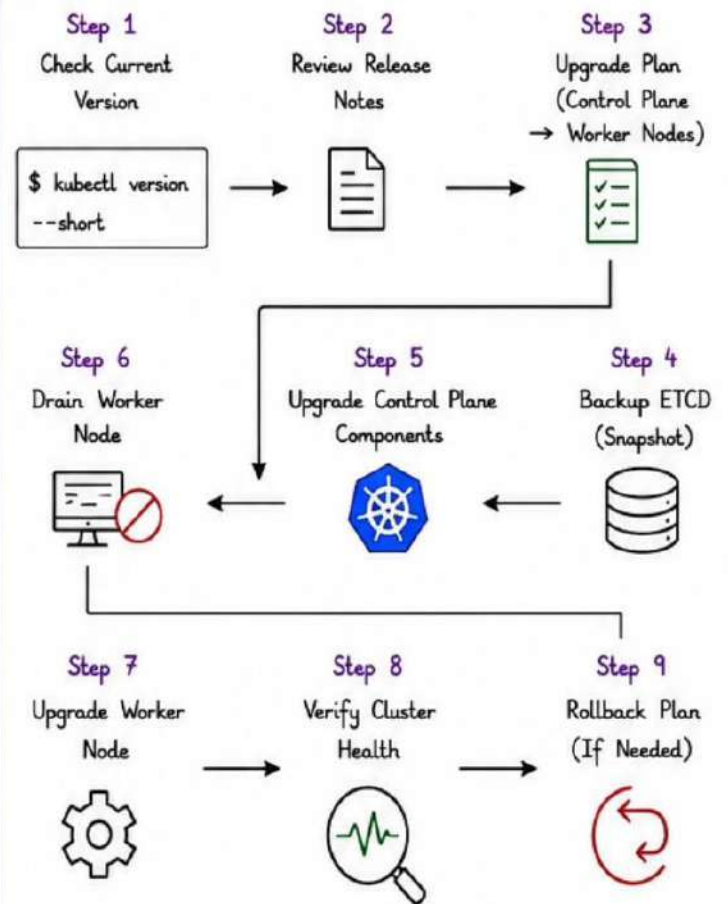
Interview Answer :

Upgrading a Kubernetes cluster is a critical task and must be performed carefully to avoid downtime. I follow a version-by-version upgrade approach as recommended by Kubernetes.

Steps I follow :

- 1 Check Current Version : `kubectl version --short`
- 2 Review Release Notes : Read release notes for the target version for breaking changes and deprecations.
- 3 Upgrade Plan : Upgrade control plane first, then worker nodes. Upgrade one node at a time.
- 4 Backup ETCD : Take a snapshot or backup of etcd for safety (if self-managed).
- 5 Upgrade Control Plane Components : Upgrade kube-apiserver, kube-controller-manager and kube-scheduler one by one.
- 6 Drain Worker Node : Drain a node to evict pods and put it in maintenance mode.
- 7 Upgrade Worker Node : Upgrade kubelet and kube-proxy, then restart services.
- 8 Verify : Verify cluster components, nodes and workloads are running fine.
- 9 Rollback Plan : If any issue occurs, rollback to the previous version using backups and auto-scaling group.

KUBERNETES UPGRADE FLOW



KEY COMMANDS

- Check version `kubectl version --short`
- Drain node `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`
- Cordon node `kubectl cordon <node>`
- Upgrade kubeadm `sudo apt-get update && sudo apt-get install -y kubeadm=<ver>`
- Upgrade node `sudo kubeadm upgrade node`
- Uncordon node `kubectl uncordon <node>`
- Check nodes `kubectl get nodes`

BEST PRACTICES

- ✓ Always upgrade one minor version at a time.
- ✓ Take etcd backup before upgrading.
- ✓ Upgrade control plane components first.
- ✓ Drain and upgrade one node at a time.
- ✓ Verify cluster health after each step.
- ✓ Test in staging environment first.
- ✓ Follow official Kubernetes upgrade guide.

COMMON MISTAKES

- ✗ Upgrading multiple nodes at once.
- ✗ Skipping release notes and breaking changes.
- ✗ Not backing up etcd.
- ✗ Ignoring addon compatibility.
- ✗ Not testing in non-production environment.
- ✗ Forcing upgrade by skipping versions.
- ✗ Not monitoring cluster after upgrade.

🕒 1-MINUTE ANSWER

I upgrade Kubernetes cluster step by step. First, I check current version and review release notes. Then I backup etcd and upgrade control plane components (API server, controller manager, scheduler) one by one. After that, I drain worker nodes and upgrade kubelet and kube-proxy. Finally, I verify cluster components and workloads. If any issue occurs, I rollback to the previous version using backups.

💡 KEY TAKEAWAYS

- Always follow version-by-version upgrade.
- Control plane components first, then worker nodes.
- Backup etcd before upgrade.
- Drain, upgrade, verify and monitor.
- Rollback plan is important for safe upgrade.

11

Q. What is a Canary Deployment ?

Explain its use case and benefits.

Interview Answer :

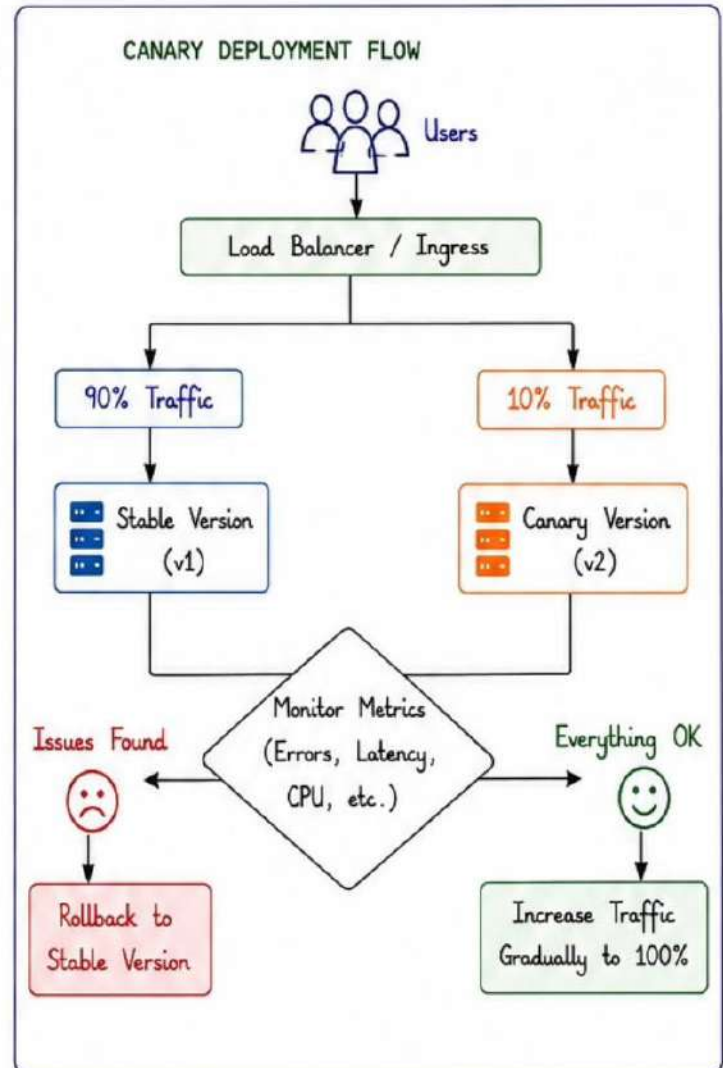
Canary Deployment is a release strategy where a new version of an application is released to a small subset of users (canary group) while the rest of the traffic continues to use the stable version. If the new version works fine, traffic is gradually increased to all users.

Use Case :

- When you want to release a new feature with minimal risk.
- To validate new version in production environment.
- To reduce the impact of bugs or performance issues.
- When you want to test with real user traffic.

Benefits :

- ✓ Reduces risk of new release.
- ✓ Early detection of issues.
- ✓ Easy rollback if something goes wrong.
- ✓ Better user experience.



TOOLS I USE

- Kubernetes (Deployments, Services, Ingress)
- Argo Rollouts (Advanced Canary)
- Istio / Linkerd (Traffic Splitting)
- Prometheus + Grafana (Monitoring)
- Datadog / New Relic (APM)

BEST PRACTICES

- ✓ Start with small traffic (5-10%).
- ✓ Monitor key metrics closely.
- ✓ Automate rollback if errors increase.
- ✓ Use feature flags for additional control.
- ✓ Gradually increase traffic after validation.

COMMON MISTAKES

- ✗ Releasing without monitoring.
- ✗ Sending too much traffic to canary.
- ✗ No proper rollback strategy.
- ✗ Ignoring error rate and latency.
- ✗ Not testing in real user environment.



1-MINUTE ANSWER

Canary Deployment means releasing a new version to a small group of users while the rest use the stable version. It monitor metrics like errors, latency and performance. If everything is good, I increase traffic gradually to 100%. If issues occur, I rollback to the stable version.



KEY TAKEAWAYS

- Canary deployment = small traffic → new version → gradual rollout.
- Reduces risk and improves release confidence.
- Monitoring and rollback are the key to success.
- Start small, validate, then scale.
- Automation helps safe and faster deployments.



Like



Comment



Share



Save

[@codewithanush](#)

12 Q. During peak traffic, the Ingress Controller is unable to route requests efficiently. How would you diagnose the issue and scale the Ingress resources ?

Interview Answer :

When Ingress Controller struggles during peak traffic, I follow a systematic approach to diagnose the root cause and scale the Ingress resources for high availability and performance.

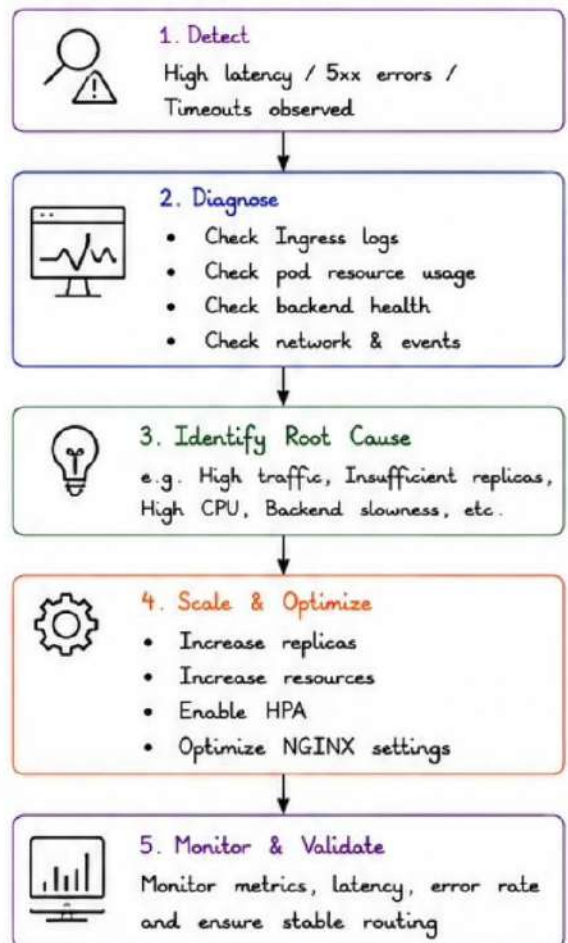
1. Diagnose the Issue

- Check Ingress Controller logs for 4xx / 5xx errors, timeouts, or reload failures.
- Monitor CPU, Memory and Network utilization of Ingress Controller pods.
- Check number of requests, latency, and backend response times using metrics.
- Verify backend service health and endpoint availability.
- Analyze network latency between Ingress and backend services.
- Check Kubernetes Events for pod restarts, OOMKills or probe failures.

2. Scale the Ingress Resources

- ✓ Increase replicas of Ingress Controller.
- ✓ Allocate more CPU and Memory to the pods.
- ✓ Enable Horizontal Pod Autoscaler (HPA) based on CPU or custom metrics (requests per second).
- ✓ Distribute traffic across multiple nodes (anti-affinity).
- ✓ Use NGINX Ingress rate limiting, caching and keep-alive connections.
- ✓ Consider sharding Ingress Controllers (by domain/path).
- ✓ Use Cloud Load Balancer for better traffic distribution.

INGRESS TROUBLESHOOTING & SCALING FLOW



TOOLS I USE

- kubectl
- NGINX Ingress Controller
- Prometheus & Grafana
- Kubernetes Dashboard
- K9s / Lens
- Cloud Load Balancer (AWS/GCP)
- Loki / ELK (for logs)

BEST PRACTICES

- ✓ Always run multiple replicas for Ingress Controller.
- ✓ Use HPA to handle traffic spikes automatically.
- ✓ Set proper resource requests & limits.
- ✓ Enable health checks and readiness/liveness probes.
- ✓ Use load balancer for global traffic distribution.
- ✓ Monitor and alert on latency & error rate.

COMMON MISTAKES

- ✗ Running single replica of Ingress Controller.
- ✗ Not setting resource limits.
- ✗ Ignoring Ingress logs and metrics.
- ✗ No HPA or autoscaling strategy.
- ✗ Backend services not optimized.
- ✗ Not testing under high load.

1-MINUTE ANSWER

When Ingress Controller fails under peak traffic, I check logs, resource usage, backend health and network latency to find the root cause. Then I increase replicas, allocate more resources, enable HPA and optimize NGINX settings. Finally, I monitor metrics to ensure stable and efficient routing.

KEY TAKEAWAYS

- Monitor everything: logs, metrics, events and backend health.
- Ingress Controller should be highly available and auto-scaled.
- Optimize resources and NGINX configuration for performance.
- Use load balancer and caching to reduce load.
- Test under load and plan for capacity.

INTERVIEWER FOLLOW-UP QUESTIONS

1. How do you enable HPA for Ingress Controller ?

Ans: I use Kubernetes HPA on CPU/Memory or custom metrics (requests per second) to automatically scale Ingress Controller pods based on load.

2. How do you reduce latency in Ingress routing ?

Ans: I enable keep-alive, caching, gzip compression, rate limiting and optimize backend services. I also use CDN or global load balancer for better performance.

3. Can we run multiple Ingress Controllers ?

Ans: Yes, we can run multiple Ingress Controllers for redundancy, different domains or path based routing and to handle higher traffic efficiently.

13

Q. How do you handle a production incident in a DevOps environment?

Walk me through your approach.

Interview Answer :

When a production incident occurs, I follow a structured approach to quickly identify, contain, and resolve the issue while minimizing impact on users and ensuring long-term prevention.

1. Detect & Acknowledge

- Alerts trigger from monitoring tools (Prometheus / CloudWatch / Datadog / Grafana).
- Acknowledge the alert immediately and validate the incident.

2. Assess & Prioritize

- Understand the impact (users affected, business impact).
- Check severity (P1 / P2 / P3) and prioritize.
- Inform the team and start incident channel (Slack / Teams).

3. Investigate & Identify Root Cause

- Check dashboards, logs (ELK / Loki), metrics and traces (X-Ray).
- Review recent deployments, infra changes and configs.
- Identify the root cause using logs, metrics and events.

4. Contain & Mitigate

- Take immediate actions to contain the issue.
- Rollback deployment / restart service / scale resources.
- Apply temporary fix or feature toggle if required.

5. Resolve & Recover

- Fix the root cause.
- Verify the service is stable and fully functional.
- Monitor closely after recovery.

6. Communicate & Document

- Keep all stakeholders informed during the incident.
- Document timeline, root cause, actions taken and impact.

7. Post-Incident Review

- Conduct post-mortem with the team.
- Identify improvement areas and update runbooks & alerts.
- Implement preventive measures to avoid recurrence.

PRODUCTION INCIDENT HANDLING FLOW



TOOLS I USE

- Monitoring : Prometheus, Grafana, CloudWatch, Datadog
- Logging : ELK Stack, Loki
- Alerting : Alertmanager, PagerDuty
- Tracing : AWS X-Ray, Jaeger
- Communication : Slack, MS Teams
- Incident Mgmt : PagerDuty, Opsgenie
- Runbook Mgmt : Confluence, Notion

BEST PRACTICES

- ✓ Detect issues early with proper monitoring & alerts.
- ✓ Keep runbooks ready for common incidents.
- ✓ Communicate clearly and frequently.
- ✓ Prioritize incidents based on business impact.
- ✓ Automate recovery wherever possible.
- ✓ Perform post-mortem and improve processes.
- ✓ Blameless culture - focus on fixing, not blaming.

COMMON MISTAKES

- ✗ Ignoring alerts or delayed response.
- ✗ Not communicating with team/stakeholders.
- ✗ Jumping to solutions without root cause.
- ✗ No runbooks or unclear documentation.
- ✗ Fixing symptoms, not the root cause.
- ✗ No post-mortem or learning from incidents.
- ✗ Not monitoring after resolution.



1-MINUTE ANSWER

When a production incident occurs, I detect and acknowledge the alert, assess the impact, and start incident channel. I investigate logs, metrics and recent changes to find the root cause. Then I contain the issue and mitigate it by rollback, restart or scaling. After fixing, I verify the service and monitor closely. I keep stakeholders updated and document everything. Finally, I conduct a post-mortem and update runbooks and alerts to prevent recurrence.



KEY TAKEAWAYS

- Quick detection and response reduce the impact.
- Root cause analysis is critical for permanent fix.
- Clear communication builds trust and reduces chaos.
- Automation and runbooks save time during incidents.
- Post-mortem and continuous improvement prevent future issues.



INTERVIEWER FOLLOW-UP QUESTIONS

1. What would you do if the issue is not fixed even after rollback?

Ans: I would check infra dependencies, database, network, and third-party services. I would scale resources, apply feature flags or implement a temporary workaround to restore service.

2. How do you decide the severity level of an incident?

Ans: I consider factors like number of users affected, business impact, service availability, revenue impact and time sensitivity to decide severity (P1/P2/P3).

3. How do you prevent similar incidents in the future?

Ans: By doing post-mortem, updating runbooks, improving monitoring, automating checks, adding alerts and testing failure scenarios.

14

Q. Explain the complete CI/CD pipeline you have implemented for a microservices application.

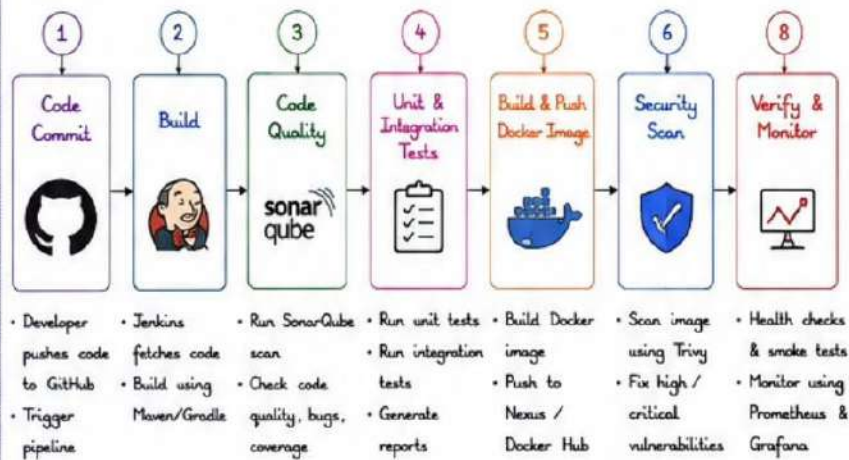
Interview Answer :

I have implemented end-to-end CI/CD pipeline for microservices using GitHub, Jenkins, Docker, SonarQube, Nexus, Kubernetes and Argo CD. The pipeline ensures code quality, security, automated testing, containerization and continuous deployment with rollback capability.

Pipeline Stages in Detail :

1. **Code Commit** → Developer pushes code to GitHub repository.
2. **Build** → Jenkins pulls code and builds the application using Maven/Gradle.
3. **Code Quality** → SonarQube scan to check code smells, bugs, vulnerabilities and test coverage.
4. **Unit & Integration Tests** → Execute automated tests and generate reports.
5. **Build & Push Docker Image** → Create Docker image and push to Nexus / Docker Hub.
6. **Security Scan** → Scan image using Trivy for vulnerabilities.
7. **Deploy to Kubernetes** → Update manifest in Git. Argo CD detects changes and deploys to Kubernetes.
8. **Verify & Monitor** → Run smoke tests, health checks and monitor using Prometheus & Grafana.

CI/CD PIPELINE FLOW



Tools & Technologies Used :

- Source Control : GitHub
- CI Server : Jenkins
- Build Tool : Maven / Gradle
- Code Quality : SonarQube
- Artifact Repo : Nexus
- Containerization : Docker
- Security Scan : Trivy
- CD Tool : Argo CD
- Orchestration : Kubernetes (EKS)
- Monitoring : Prometheus, Grafana

Key Highlights :

- ✓ Automated build, test and deployment
- ✓ Code quality and security gates
- ✓ Immutable Docker images
- ✓ GitOps with Argo CD
- ✓ Zero-downtime deployment
- ✓ Easy rollback to previous version
- ✓ Full monitoring & alerting

BEST PRACTICES

- ✓ Keep pipeline fast and fail early.
- ✓ Run tests in parallel to reduce time.
- ✓ Use caching for dependencies.
- ✓ Scan code & images for security.
- ✓ Use version tags for Docker images.
- ✓ Store all configs in Git (GitOps).
- ✓ Monitor pipeline and set alerts.

COMMON MISTAKES

- ✗ Skipping tests to save time.
- ✗ Hardcoding credentials in pipeline.
- ✗ Not scanning images for vulnerabilities.
- ✗ Manual approvals at every stage.
- ✗ Deploying directly to production.
- ✗ Not having rollback strategy.
- ✗ Poor logging and monitoring.

TOOLS I USE

- Jenkins Pipeline (Declarative)
- GitHub Webhooks
- Docker Multi-stage Builds
- SonarQube Quality Gates
- Trivy Image Scanning
- Argo CD for GitOps
- Prometheus + Grafana Dashboards



1-MINUTE ANSWER

I follow CI/CD pipeline starting from code commit to deployment. Code is pushed to GitHub, Jenkins builds the code, runs tests and SonarQube scan. Then Docker image is built, scanned and pushed to Nexus. Argo CD deploys changes to Kubernetes. Finally, we verify with smoke tests and monitor using Prometheus & Grafana with rollback option.



KEY TAKEAWAYS

- Automate everything to reduce human errors.
- Quality and security checks should be part of pipeline.
- GitOps ensures traceability and consistency.
- Monitoring and rollback are critical for production.
- Always optimize pipeline for speed and reliability.



INTERVIEWER FOLLOW-UP QUESTIONS

1. How do you handle rollback in your pipeline ?

Ans : I keep previous Docker image tags and Kubernetes manifests. If new deployment fails health checks, Argo CD rolls back to previous stable version automatically.

2. How do you manage secrets in pipeline ?

Ans : I use Jenkins credentials for CI/CD secrets, and AWS Secrets Manager / Kubernetes Secrets for application config and sensitive data.

3. How do you ensure zero downtime deployment ?

Ans : I use rolling updates with readiness & liveness probes in Kubernetes. If something goes wrong, it automatically rolls back.